



Offerbook: NFT Review

Security Review

Cantina Managed review by:

S3v3ru5, Security Researcher

Jiri123, Associate Security Researcher

May 21, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	5
3.1	Medium Risk	5
3.1.1	NFT repay sends collateral to borrower wallet instead of borrower escrow, breaking escrow design and the natural repay flow	5
3.2	Low Risk	6
3.2.1	Non-fungible principal offers with <code>Collection</code> and <code>FirstVerifiedCreator</code> filters are unfillable	6
3.2.2	Persistent MPL Core permanent delegates let borrowers steal locked NFT collateral after fill	7
3.2.3	Lender receives borrower's ATA rent when filling a pNFT collateral offer	8
3.2.4	Borrower receives lender's <code>loan_vault</code> rent when repaying an NFT loan	9
3.2.5	<code>RefereeReward</code> event sets <code>referee</code> to the User PDA in principal-offer fills	10
3.2.6	Token-2022 <code>TransferHook</code> restriction can be bypassed	10
3.2.7	Standalone Core NFTs cannot be filled as collateral	11
3.3	Informational	11
3.3.1	Validations in escrow deposit/withdraw do not protect offer creation	11
3.3.2	Closing borrower escrow ATA during pNFT fill forces recreation at repay	13
3.3.3	Code Improvements	13

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Apr 10th to Apr 22nd the Cantina team conducted a review of [offerbook-program](#) on commit hash [80ec73bd](#). The team identified a total of **11** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	7	5	2
Gas Optimizations	0	0	0
Informational	3	2	1
Total	11	8	3

2.1 Scope

The security review had the following components in scope for [offerbook-program](#) on commit hash [80ec73bd](#):

```
programs/offerbook/src
├── constants.rs
├── error.rs
├── events.rs
├── lib.rs
├── utils.rs
├── instructions
│   ├── cancel_offer.rs
│   ├── claim_fee.rs
│   ├── create_user.rs
│   ├── init.rs
│   ├── mod.rs
│   ├── update_config.rs
│   └── fungible
│       ├── claim_token_loan.rs
│       ├── create_token_collateral_offer.rs
│       ├── create_token_principal_offer.rs
│       ├── escrow_token_deposit.rs
│       ├── escrow_token_withdraw.rs
│       ├── fill_token_collateral_offer.rs
│       ├── fill_token_principal_offer.rs
│       ├── mod.rs
│       └── repay_token_loan.rs
├── non_fungible
│   ├── claim_non_fungible_loan.rs
│   ├── create_non_fungible_collateral_offer.rs
│   ├── create_non_fungible_principal_offer.rs
│   ├── fill_non_fungible_collateral_offer.rs
│   ├── fill_non_fungible_principal_offer.rs
│   ├── mod.rs
│   ├── repay_non_fungible_loan.rs
│   └── classic
│       ├── escrow_classic_nft_deposit.rs
│       ├── escrow_classic_nft_withdraw.rs
│       └── mod.rs
├── mpl_core
│   ├── escrow_core_nft_deposit.rs
│   └── escrow_core_nft_withdraw.rs
```

```
├── mod.rs
├── programmable
│   ├── escrow_programmable_nft_deposit.rs
│   ├── escrow_programmable_nft_withdraw.rs
│   └── mod.rs
└── state
    ├── asset.rs
    ├── config.rs
    ├── loan.rs
    ├── mod.rs
    ├── offer.rs
    └── user.rs
```

3 Findings

3.1 Medium Risk

3.1.1 NFT repay sends collateral to borrower wallet instead of borrower escrow, breaking escrow design and the natural repay flow

Severity: Medium Risk

Context: [repay_non_fungible_loan.rs#L331](#)

Description: `repay_non_fungible_loan` sets the destination owner of the `ProgrammableNft` collateral transfer to the borrower's wallet (`signer`), not the borrower's User PDA escrow (`signer_user`). This contradicts the protocol's escrow-based design pattern and causes repay to fail when the frontend passes the natural account derived from the same naming convention used everywhere else in the protocol.

In `RepayProgrammableNftAccounts::return_collateral` :

```
destination_token: &self.borrower_collateral_escrow.to_account_info(),
destination_owner: &repay_accounts.signer.to_account_info(),
```

In `RepayCoreNftAccounts::return_collateral` :

```
authority: Some(&loan),
new_owner: &signer,
```

Metaplex's `TransferV1` validates that `destination_token.owner == destination_owner` , so the destination token account must have the borrower's wallet as its authority.

The constraint on `borrower_collateral_escrow` in the NFT repay structs is `token::mint` and `token::token_program` only, with no authority constraint. The fungible repay path enforces `associated_token::authority = signer_user` on the same-named field, which constrains the destination to the borrower's User PDA ATA.

The naming `borrower_collateral_escrow` matches the field used elsewhere in the protocol where it refers to the User PDA's ATA:

- `borrower_collateral_escrow` in `fill_non_fungible_*_offer` (sourced from a `borrower_user` - authorized account at fill).
- `borrower_collateral_escrow` in `repay_token_loan` (constrained to `associated_token::authority = signer_user`).

A frontend integrating with the NFT repay path based on this naming and the fungible repay precedent would pass the borrower's User PDA ATA. The `validate_token` check inside `TransferV1` then fails because the User PDA ATA's authority is `signer_user` , not `signer` . Repay reverts. The borrower cannot repay until the integrator passes a wallet-owned account instead, forcing the borrower to default.

This also breaks the protocol's escrow-based design. NFTs are deposited into User PDA escrows (`escrow_programmable_nft_deposit` , `escrow_core_nft_deposit`) and locked into loan vaults at fill. The intended flow is for them to return to the User PDA escrow on repay so the borrower can immediately reuse them in another offer. Sending them to the borrower's wallet forces a re-deposit (extra transaction, gas, ATA rent, token record rent for pNFTs) before the NFT can be used as collateral again.

After the transfer, `repay_non_fungible_loan` emits an `EscrowDeposit` event. `EscrowDeposit` claims a deposit into escrow, but the NFT actually went to the borrower's wallet. Off-chain indexers tracking escrow balances by parsing this event will record a phantom deposit that doesn't correspond to any real escrow change.

For comparison, the fungible repay path correctly enforces escrow as the destination via `associated_token::authority = signer_user` on `borrower_collateral_escrow` and naturally signs the transfer to land collateral in the borrower's User PDA escrow.

The claim NFT path has an analogous structure, both pNFT and Core claim send the NFT to the lender's wallet (`signer`) rather than the lender's User PDA escrow (`signer_user`). This is less critical than the repay case because claiming a defaulted NFT to a personal wallet is a reasonable user choice (the lender

now owns it outright and may want it outside the protocol). The fungible claim path, in contrast, sends to the lender's User PDA escrow.

Exploit Scenario:

1. A borrower deposits a pNFT into their User PDA escrow via `escrow_programmable_nft_deposit`.
2. The borrower creates a collateral offer. A lender fills it. The pNFT is locked in `loan_vault`.
3. The borrower attempts to repay before expiry. The frontend, following the protocol's naming and fungible-repay precedent, passes the borrower's User PDA ATA as `borrower_collateral_escrow`.
4. The transfer reverts inside `TransferV1` because the destination's authority (User PDA) does not match `destination_owner` (signer wallet).
5. The borrower cannot repay until the frontend works around this by passing a wallet-owned account.
6. Even when working around it, the NFT lands in the borrower's wallet, not their User PDA escrow. To use it as collateral again, the borrower must re-deposit, paying extra gas and rent.

If repay cannot complete, the borrower is forced to default once the loan expires. The lender claims the collateral, but only because of an integration bug, not because the borrower failed to provide funds.

Recommendation: Update both NFT repay paths to use `signer_user` as the destination owner, matching the protocol's escrow-based design and the fungible repay path:

- In `RepayProgrammableNftAccounts::return_collateral`, set `destination_owner: &repay_accounts.signer_user.to_account_info()`.
- In `RepayCoreNftAccounts::return_collateral`, set `new_owner: &signer_user`.
- Add `associated_token::authority = signer_user` constraint on `borrower_collateral_escrow` to enforce the User PDA's ATA.

For consistency with the existing `escrow_programmable_nft_deposit` flow, the User PDA's ATA must exist or be initializable at repay time.

For the claim NFT paths, consider whether sending to the lender's User PDA escrow (matching fungible claim) or the lender's wallet (current behavior) is the intended design.

Offerbook: Fixed in commit [df0e832](#).

Cantina Managed: Fixed. The borrower's collateral escrow account is verified to be the canonical ATA of `signer_user` via runtime derivation in `RepayClassicNftAccounts::return_collateral` and `RepayProgrammableNftAccounts::return_collateral`. The pNFT `destination_owner` and Core `new_owner` parameters are updated to `signer_user`. The check is performed at runtime since `signer_user` resides in a separate context struct and is not accessible to Anchor constraints in the wrapper.

3.2 Low Risk

3.2.1 Non-fungible principal offers with `Collection` and `FirstVerifiedCreator` filters are unfillable

Severity: Low Risk

Context: [create_non_fungible_principal_offer.rs#L93-L100](#)

Description: The `create_nft_principal_offer` instruction accepts a `NftCollateral` enum that includes `Collection` and `FirstVerifiedCreator` variants. When these variants are used, `into_asset_and_filter()` sets the offer's collateral to `Asset::None` and stores the filter criteria in `offer.filter`:

```
// create_non_fungible_principal_offer.rs:93-99
NftCollateral::Collection { collection } => (
    Asset::None,
    AssetFilter::Collection(CollectionFilter::new(collection)),
),
NftCollateral::FirstVerifiedCreator { creator } => (
    Asset::None,
    AssetFilter::FirstVerifiedCreator(FirstVerifiedCreatorFilter::new(creator)),
),
```

However, when a borrower attempts to fill such an offer via `fill_non_fungible_principal_offer`, two independent checks block execution:

1. The `nft_wrapper()` function dispatches on `offer.collateral`. Since it is `Asset::None`, it falls into the default arm and returns `InvalidCollateral`.
2. Even if `nft_wrapper` were bypassed, `offer.validate_asset()` performs strict equality (`collateral == &self.collateral`), which would fail because the actual NFT asset (e.g. `Asset::ClassicNft`) never equals `Asset::None`.

The `offer.filter` field is never consulted during fill validation, so the filter data is written but unused. Offers created with these variants consume rent from the creator and are only recoverable via `cancel_offer`.

Exploit Scenario:

1. A lender calls `create_nft_principal_offer` with `NftCollateral::Collection { collection: <xyz> }`, paying rent for the offer account.
2. A borrower holding an NFT from collection `<xyz>` attempts to call `fill_non_fungible_principal_offer`.
3. The transaction reverts with `InvalidCollateral` at the `nft_wrapper` dispatch.
4. The offer sits on-chain unfillable until the lender cancels it and reclaims rent.

Recommendation: Either remove `Collection` and `FirstVerifiedCreator` from the `NftCollateral` enum if filter-based offers are not intended for V1, or implement filter-aware validation in the fill path that:

- Extends `nft_wrapper()` to accept `Asset::None` and determine NFT type from `remaining_accounts`.
- Replaces the strict `validate_asset` equality check with filter-based validation (e.g. verifying the NFT's on-chain metadata collection field matches `offer.filter`).

Offerbook: Fixed in commit [ee4a74a](#).

Cantina Managed: Fixed. `fill_non_fungible_principal_offer` now supports filter-based offers. The `NftWrapper` trait is extended with `validate_filter` and `get_collection` methods, implemented for Classic, Programmable, and Core NFT wrappers. A new `validate_offer_assets` function dispatches to filter validation when `offer.collateral == Asset::None`, and the `nft_wrapper` dispatcher handles `Asset::None` by attempting to deserialize remaining accounts as Programmable, Classic, or Core in turn.

3.2.2 Persistent MPL Core permanent delegates let borrowers steal locked NFT collateral after fill

Severity: Low Risk

Context: `fill_non_fungible_principal_offer.rs#L231-L240`

Description: The protocol accepts MPL Core NFTs as loan collateral by recording only the Core asset pubkey.

During fill, the NFT is transferred to the loan PDA, but the program does not inspect or reject MPL Core asset/collection plugins such as `PermanentTransferDelegate` or `PermanentBurnDelegate`. These authorities can survive ownership changes and remain valid even after the asset owner becomes the loan PDA.

A borrower can therefore use a Core NFT with a permanent transfer or burn delegate pointing to an attacker-controlled authority, receive principal after fill, then call MPL Core directly to transfer or burn the locked collateral outside Offerbook. The lender's later claim fails because the loan PDA no longer exclusively controls the collateral.

Recommendation: Reject MPL Core collateral that has asset-level or collection-level `PermanentTransferDelegate` or `PermanentBurnDelegate` plugins, or any other non-owner authority that can move or destroy the asset after lock.

Offerbook: We want to keep the option for users to borrow or lend against collections that have a `PermanentTransferDelegate` or `PermanentBurnDelegate`. The regular `TransferDelegate` is reset upon transfer.

If a collection is compromised, the resulting loss is outside our threat model.

Cantina Managed: Acknowledged.

3.2.3 Lender receives borrower's ATA rent when filling a pNFT collateral offer

Severity: Low Risk

Context: [fill_non_fungible_collateral_offer.rs#L351-L360](#)

Description: When a lender fills a borrower's Programmable NFT collateral offer, `ProgrammableNftAccounts::lock_nft` closes the borrower's collateral escrow ATA and sends the rent to the lender instead of the borrower. The borrower originally paid for this ATA during `escrow_programmable_nft_deposit` (via `init_if_needed`), so the rent transfer is a direct loss to the borrower.

```
close_account(  
    CpiContext::new(  
        self.collateral_token_program.to_account_info(),  
        CloseAccount {  
            account: self.borrower_collateral_escrow.to_account_info(),  
            destination: fill_offer_accounts.signer.to_account_info(), // lender  
            authority: fill_offer_accounts.borrower_user.to_account_info(),  
        },  
    ).with_signer(&[seeds]),  
)?;
```

In this instruction, `signer` is the lender (the filler). The destination of the closed ATA's rent is the lender, not the borrower who originally paid for it.

Exploit Scenario:

1. A borrower deposits a pNFT into their escrow via `escrow_programmable_nft_deposit`, paying rent to create the ATA.
2. The borrower creates a pNFT collateral offer and a lender fills it.
3. `lock_nft` transfers the pNFT to the loan vault and closes the borrower's ATA, sending the rent to the lender.
4. The borrower receives the loan principal but loses the ATA rent permanently regardless of whether they repay or default.

Recommendation: Set `destination` to the borrower instead of the signer.

Offerbook: Fixed in commit [c1c8f80](#).

Cantina Managed: Fixed. In `ProgrammableNftAccounts::lock_nft`, the `close_account` destination for the borrower's collateral escrow ATA is changed from `fill_offer_accounts.signer.to_account_info()` (the lender) to `fill_offer_accounts.borrower.to_account_info()`, ensuring the ATA rent returns to the borrower who originally paid for it at deposit time.

3.2.4 Borrower receives lender's `loan_vault` rent when repaying an NFT loan

Severity: Low Risk

Context: [repay_non_fungible_loan.rs#L276](#)

Description: In `repay_non_fungible_loan`, both `RepayClassicNftAccounts::return_collateral` and `RepayProgrammableNftAccounts::return_collateral` close the `loan_vault` token account with `destination` set to `repay_accounts.signer`. The signer is always the borrower in repay, but the rent for the loan vault was paid by `loan.creator` at fill time, which is the lender when the offer was a collateral-side offer.

The same pattern exists in `claim_non_fungible_loan`: `ClaimClassicNftAccounts::claim_collateral` and `ClaimProgrammableNftAccounts::claim_collateral` close `loan_vault` with `destination = signer` (the lender). When `loan.creator = borrower` (principal-side offer fill), the lender receives the rent the borrower originally paid.

Exploit Scenario:

1. A borrower creates an NFT collateral offer.
2. A lender fills the offer via `fill_non_fungible_collateral_offer`. The lender pays for the `loan_vault` token account and is recorded as `loan.creator`.
3. The borrower repays the loan via `repay_non_fungible_loan`. The repay path closes `loan_vault` with destination set to the signer.
4. The borrower receives the rent the lender originally paid.

Recommendation: Mirror the fungible token repay path: compute `rent_destination` based on `loan.creator` and use it as the `CloseAccount::destination`. This ensures the rent returns to whoever paid for the vault.

Offerbook: Fixed in commits [28fbba2](#) (repay) and [983ac64](#) (claim).

Cantina Managed: Fixed. The `loan_vault` close destination in both NFT repay and NFT claim paths is now conditional on `loan.creator`:

- In `RepayClassicNftAccounts::return_collateral` and `RepayProgrammableNftAccounts::return_collateral`, the destination is set to `signer` when `loan.creator == signer.key()` and to `lender` otherwise.
- In `ClaimClassicNftAccounts::claim_collateral` and `ClaimProgrammableNftAccounts::claim_collateral`, the destination is set to `signer` when `loan.creator == signer.key()` and to `borrower` otherwise.

3.2.5 RefereeReward event sets referee to the User PDA in principal-offer fills

Severity: Low Risk

Context: `fill_non_fungible_principal_offer.rs#L607`

Description: The `RefereeReward` event's `referee` field is intended to identify the wallet eligible to claim the referee rebate. The frontend reads these events and allows the corresponding wallet to claim the rebate. Across most instructions, `referee` is set to the user's wallet pubkey, but in two instructions, `fill_non_fungible_principal_offer` and `fill_token_principal_offer`, it is set to the User PDA address instead.

The frontend computes referee rebates by matching `referee` to wallet pubkeys. Events that emit the User PDA instead will match to the User PDA, and users cannot claim them.

Exploit Scenario:

1. A borrower with a registered referrer fills a principal offer via `fill_non_fungible_principal_offer`.
2. The protocol fee is non-zero and the referee reward is non-zero, so the `RefereeReward` event is emitted.
3. The event's `referee` field is the borrower's User PDA address rather than the borrower's wallet.
4. The frontend's referee rebate distribution only allows the User PDA address to claim the fees.

Recommendation: Change both emissions to use the wallet pubkey:

- In `fill_token_principal_offer`, set `referee: ctx.accounts.signer.key()` (the borrower).
- In `fill_non_fungible_principal_offer`, set `referee: ctx.accounts.signer.key()`.

This matches the convention used by every other emission of `RefereeReward`.

Offerbook: Fixed in commit [2042f57](#).

Cantina Managed: Fixed. The `RefereeReward` event's `referee` field is set to `ctx.accounts.signer.key()` in both `fill_token_principal_offer` and `fill_non_fungible_principal_offer`, matching the convention used by every other `RefereeReward` emission.

3.2.6 Token-2022 TransferHook restriction can be bypassed

Severity: Low Risk

Context: `utils.rs#L34-L37`

Description: Offerbook attempts to restrict fungible Token-2022 collateral by rejecting mints whose `TransferHook.program_id` is currently populated. However, this check can be bypassed because a Token-2022 mint can already contain a mutable `TransferHook` extension with a live hook authority while its hook program is still unset.

In that state, the mint appears safe to the current validation because `program_id` is default. After the loan is funded and collateral has been escrowed, the hook authority can set `TransferHook.program_id` to a hook program that rejects future transfers.

This bypasses the intended `TransferHook` restriction: the collateral mint passes validation first, then becomes transfer-gated only after the protocol has accepted it.

Recommendation: Reject any mutable `TransferHook` state for fungible collateral. At minimum, do not rely solely on `TransferHook.program_id`; also reject mints where the `TransferHook` extension is present with a live authority, even if the hook program is currently unset.

Offerbook: Acknowledged. We accept this risk. Restricting the authority field would block xStocks tokens, which we want to support.

Cantina Managed: Acknowledged.

3.2.7 Standalone Core NFTs cannot be filled as collateral

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: `CoreNftAccounts` in `fill_non_fungible_collateral_offer` and `PrincipalCoreNftAccounts` in `fill_non_fungible_principal_offer` declare `collection` as a required `UncheckedAccount`. In MPL Core, an asset's `update_authority` field can be `UpdateAuthority::Collection(pubkey)` or `UpdateAuthority::Address(pubkey)`. The latter case is a standalone asset with no associated collection account.

The escrow, repay, and claim Core paths declare `collection` as `Option<UncheckedAccount>`. The fill paths are the only Core paths that require `collection` unconditionally. Standalone Core assets have no valid `collection` account to pass at fill time and therefore cannot be used as collateral.

Exploit Scenario:

1. Bob owns a standalone Core asset.
2. User deposits the asset into their User PDA escrow via `escrow_core_nft_deposit` and creates an NFT collateral offer.
3. A lender attempts to fill the offer via `fill_non_fungible_collateral_offer`. There is no valid `collection` account for the asset.
4. The fill cannot be constructed. The asset remains unusable as collateral.

Recommendation: Make `collection` optional in `CoreNftAccounts` and `PrincipalCoreNftAccounts` to match the other Core paths. Update the corresponding `lock_nft` implementations to pass `collection.as_ref().map(|c| c.to_account_info())` to `CoreTransferV1CpiAccounts::collection`.

Offerbook: Fixed in commit [b0c123b](#).

Cantina Managed: Fixed. `CoreNftAccounts` and `PrincipalCoreNftAccounts` now declare `collection` as `Option<UncheckedAccount<'info>>`, enabling standalone Core assets to be used as collateral. The corresponding `lock_nft` implementations pass `collection.as_ref().map(|c| c.to_account_info())` to `CoreTransferV1CpiAccounts::collection`.

3.3 Informational

3.3.1 Validations in escrow deposit/withdraw do not protect offer creation

Severity: Informational

Context: [escrow_classic_nft_deposit.rs#L39](#)

Description: The escrow deposit and withdraw instructions perform validations on the deposited assets. However, these checks do not serve as a gatekeeping mechanism for offer creation. The escrow token accounts are standard associated token accounts (ATAs) owned by the User PDA. Anyone can create these ATAs off-chain and transfer tokens or NFTs into them without ever invoking the program's deposit instructions. This means the mint and asset validations present in escrow instructions can be bypassed

entirely, and an offer can be created referencing any mint regardless of whether it passed the escrow checks.

The following validations exist only in escrow instructions and are bypassable:

Classic NFT escrow deposit/withdraw (`escrow_classic_nft_deposit.rs` , `escrow_classic_nft_withdraw.rs`):

- `nft_mint.supply == 1 && nft_mint.decimals == 0` - validates the mint is an NFT.
- `nft_metadata.key() == Metadata::find_pda(&nft_mint.key()).0` - validates metadata account exists and derives correctly.
- `nft_edition.key() == MasterEdition::find_pda(&nft_mint.key()).0` - validates master edition exists and derives correctly.

Programmable NFT escrow deposit/withdraw (`escrow_programmable_nft_deposit.rs` , `escrow_programmable_nft_withdraw.rs`):

- `nft_mint.supply == 1 && nft_mint.decimals == 0` - validates the mint is an NFT.
- `nft_metadata.key() == Metadata::find_pda(&nft_mint.key()).0` - validates metadata account.
- `nft_edition.key() == MasterEdition::find_pda(&nft_mint.key()).0` - validates master edition.

Core NFT escrow deposit/withdraw (`escrow_core_nft_deposit.rs` , `escrow_core_nft_withdraw.rs`):

- `Account<'info, BaseAssetV1>` type deserialization - validates the account is a valid MPL Core asset.

The NFT create offer instructions (`create_nft_collateral_offer` , `create_nft_principal_offer`) accept collateral asset identifiers as instruction data (pubkeys inside `NftCollateralAsset` / `NftCollateral` enums) with no on-chain validation that the referenced mint or asset is a valid NFT. While the fill path does re-validate NFT properties through remaining account structs, invalid offers can still be created and sit on-chain unfillable.

Exploit Scenario:

1. A borrower creates an ATA for the User PDA off-chain and transfers a fungible token (supply > 1) into it.
2. The borrower calls `create_nft_collateral_offer` with `NftCollateralAsset::ClassicNft { mint: <fungible_token_mint>, token_program: <...> }`.
3. The offer is created successfully since no on-chain validation of the mint occurs at offer creation.
4. A lender attempts to fill the offer. The fill instruction's `ClassicNftAccounts` constraint `nft_mint.supply == 1 && nft_mint.decimals == 0` rejects the transaction.
5. The offer is unfillable, wasting rent until cancelled.

Recommendation: Move the asset validations from the escrow deposit/withdraw instructions to the create offer instructions where the assets are first referenced. This ensures invalid assets are rejected at offer creation time rather than at fill time.

Offerbook: The check cannot be performed at offer creation because the NFT accounts are not passed. The check has been moved to fill time in commit [a794a5f](#).

Cantina Managed: Fixed. The commit adds the `supply == 1` and `decimals == 0` validation to `nft_mint` in the Programmable NFT fill paths. The remaining escrow validations are already present at fill time.

It remains possible to create an offer that does not pass the escrow creation validations, but such an offer is unfillable. All escrow validations are replicated at fill time.

3.3.2 Closing borrower escrow ATA during pNFT fill forces recreation at repay

Severity: Informational

Context: `fill_non_fungible_collateral_offer.rs#L351`

Description: `ProgrammableNftAccounts::lock_nft` and `PrincipalProgrammableNftAccounts::lock_nft` close the borrower's collateral escrow ATA after transferring the pNFT to the loan vault. This design has two downsides without a clear benefit:

1. The repay paths in `repay_non_fungible_loan` require `borrower_collateral_escrow` to exist when returning the NFT. They do not use `init_if_needed`, so the borrower must manually recreate the ATA before repay. This adds an extra instruction or transaction to the repay flow.
2. Closing and recreating the ATA is rent-neutral for the borrower but moves the rent burden around. The borrower paid for the ATA at deposit, gets the rent back at fill, then pays again to recreate the ATA at repay. The net cost is the same except for transaction fees.

Recommendation: Remove the `close_account` calls from `ProgrammableNftAccounts::lock_nft`, and `PrincipalProgrammableNftAccounts::lock_nft` and let the borrower close the ATA explicitly via `escrow_programmable_nft_withdraw`.

Offerbook: Acknowledged. We prefer to close the ATA considering the event of a liquidation.

Cantina Managed: Acknowledged.

3.3.3 Code Improvements

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description:

1. Add `nft_mint.supply == 1 && nft_mint.decimals == 0` constraint to `ProgrammableNftAccounts` in `fill_non_fungible_collateral_offer.rs` and `PrincipalProgrammableNftAccounts` in `fill_non_fungible_principal_offer.rs`. While the Metaplex program enforces pNFT supply of 1, adding the constraint keeps validation consistent with all other NFT instruction paths (escrow, repay, claim, and Classic NFT fill).
2. Validate the NFT's on-chain `metadata.token_standard` against the declared `Asset` variant at offer creation or fill time. Currently the program does not differentiate Classic and Programmable NFTs on-chain and trusts the caller's declaration. Since `TransferV1Cpi` handles both standards, a Classic NFT declared as `ProgrammableNft` will transfer successfully through the entire loan lifecycle, storing an incorrect asset type that misleads off-chain indexers and event consumers.
3. Remove dead code related to `OfferOrigin` :
 - Remove the `OfferOrigin` enum and its `Pod / Zeroable` impls in `offer.rs`.
 - Remove the unused `InvalidOfferOrigin` error variant in `error.rs`.
 - Update the `Offer::MAX_SIZE` comment to drop the `1 // origin` line (the byte is already part of `_padding`).
4. Remove `CollectionWithAttributeFilter` from the `AssetFilter` enum or add a corresponding variant to `NftCollateral`. Unlike `Collection` and `FirstVerifiedCreator` which can at least be created, `CollectionWithAttributeFilter` has no offer creation path and is entirely unreachable dead code.

5. Add `associated_token::authority = borrower_user` constraint to `borrower_collateral_escrow` in `Nft` fill `remaining_accounts` structs (`ClassicNftAccounts`, `ProgrammableNftAccounts`, `PrincipalClassicNftAccounts`, `PrincipalProgrammableNftAccounts`). Currently these only check `token::mint`, so the filler can pass any `borrower_user`-owned token account holding the NFT — not just the escrow ATA. The CPI transfer still succeeds since `borrower_user` must sign, but the emitted `EscrowWithdraw` event would incorrectly claim a withdrawal from escrow when the NFT may never have been deposited through the escrow instructions.
6. Remove dead error variants from `error.rs` :
 - `DuplicateResources` .
 - `AdminArrayFull` .
 - `InvalidOfferOrigin` .
7. Remove `#[account(mut)]` from `authorization_rules` in all six pNFT instruction structs (`EscrowProgrammableNftDepositContext`, `EscrowProgrammableNftWithdrawContext`, `ProgrammableNftAccounts`, `PrincipalProgrammableNftAccounts`, `RepayProgrammableNftAccounts`, `ClaimProgrammableNftAccounts`). The Metaplex Auth Rules program treats rule set accounts as read-only during transfer validation, so `mut` is incorrect. Marking it writable forces callers to declare the account as writable unnecessarily and prevents parallel transaction execution on the same rule set.
8. Validate `token_program == *mint.owner` at NFT offer creation in `create_nft_principal_offer` and `create_nft_collateral_offer`. The `NftCollateral` and `NftCollateralAsset` enums accept `token_program` as raw instruction data with no validation, so a creator can declare an inconsistent `mint` / `token_program` pair and produce an offer that is unfillable at fill time when the `token::token_program = collateral_token_program` constraint rejects the mismatch. Requiring the mint account at offer creation and checking `token_program == *mint.owner` catches the mistake early and avoids wasted rent.
9. Add `init_if_needed` to `lender_collateral_escrow` in `claim_token_loan`. If the lender has never held the collateral mint in their User PDA escrow, the ATA does not exist and the claim reverts with `AccountNotInitialized`. The lender must create the ATA manually before retrying.
10. Emit events for all admin-mutating operations. Currently `init`, `update_config` (16 variants), and `claim_fee` emit zero events, while every user-facing operation is instrumented. Off-chain consumers must poll `Config` state to detect admin actions. Since `update_config` dispatches a single `UpdateConfigAction` enum, a single `ConfigUpdated` event carrying the action variant and its arguments suffices instead of one event per variant.
11. Replace the `panic!` in `compute_claim_amounts` with a structured `OfferbookError`. Panics produce a program abort rather than a typed error. The path is currently unreachable since principal is always Token under the V1 USDC enforcement, but a future refactor that loosens this assumption would hit an abort instead of a clean error.
12. Emit a `UserCreated` event in `create_user`. The instruction creates a new User PDA and optionally increments a referrer's `referral_count`, but emits no events. All other user-facing operations emit events. Off-chain consumers building user graphs or referral trees cannot detect new registrations through the event stream.
13. Close the loan account at repay or claim, or add a separate `close_loan` instruction. Loan accounts are never closed and persist on-chain indefinitely after settlement, locking ~0.0057 SOL of rent per loan that the filler cannot reclaim.

Offerbook: Items 1, 2, 3, 5, 6, 7, and 11 have been addressed in the [code_impro](#) branch. Items 4, 8, 10, and 12 will not be implemented. Item 9 is handled on the backend during transaction construction by

including the ATA creation instruction when needed. Item 13 will be addressed in a future release through a dedicated close-loan instruction; for now, loan accounts are kept on-chain for historical purposes.

Cantina Managed: Partially fixed. Implementations of suggestions 1, 2, 3, 5, 6, 7, and 11 are verified.